In [19]:
```
import nltk, random
nltk.download('twitter_samples')
from nltk.corpus import twitter_samples
```

```
[nltk_data] Downloading package twitter_samples to C:\Users\Ujjwal
[nltk_data]     Karki\AppData\Roaming\nltk_data...
[nltk_data]   Package twitter_samples is already up-to-date!
```

In [20]:
```
pos = twitter_samples.strings('positive_tweets.json')
neg = twitter_samples.strings('negative_tweets.json')
```

In [21]:
```
data = [(t, 1) for t in pos] + [(t, 0) for t in neg]
random.shuffle(data)
```

In [22]:
```
train, test = data[:8000], data[8000:]
```

In [23]:
```
import re
def clean(t):
  # replace https with " "
  t = re.sub(r"http\S+|@\w+", " ", t)
  t = re.sub(r"[^a-zA-Z\s]", " ", t)
  return t.lower().split()
```

In [24]:
```
from collections import Counter
counts = Counter(w for t, _ in train for w in clean(t))
```

In [25]:
```
vocab = {"<pad>": 0, "unk": 1}
for w, _ in counts.most_common(8000):
  vocab[w] = len(vocab)
```

In [26]:
```
import torch
from torch.utils.data import DataLoader, TensorDataset
```

In [27]:
```
def encode(t, maxlen=30):
  ids = [vocab.get(w, 1) for w in clean(t)][:maxlen]
  return ids + [0]*(maxlen - len(ids))
```

In [28]:
```
Xtr = torch.tensor([encode(t) for t, _ in train])
ytr = torch.tensor([l for _, l in train])
```

In [29]:
```
train_loader = DataLoader(TensorDataset(Xtr, ytr), batch_size = 32, shuffle=True
```

In [30]:
```
import torch.nn as nn
```

In [31]:
```
class SentimentRNN(nn.Module):
  def __init__(self, vocab_size, emb=64, hidden=128):
    super().__init__()
    self.embedding = nn.Embedding(vocab_size, emb)
    self.rnn = nn.RNN(emb, hidden, batch_first=True)
    self.fc = nn.Linear(hidden, 2)

  def forward(self, x):
    x = self.embedding(x)
    x, _ = self.rnn(x)
    return self.fc(x[:, -1]) # last hidden memory
```

In [32]:
```
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using {device} device")
```

```
Using cuda device
```

In [33]:
```
rnn = SentimentRNN(len(vocab)).to(device)
optimizer = torch.optim.Adam(rnn.parameters(), weight_decay = 0.0001)
loss_fn = nn.CrossEntropyLoss()
```

In [34]:
```
for epoch in range(1500):
  rnn.train()
  for xb, yb in train_loader:
    xb, yb = xb.to(device), yb.to(device)
    pred = rnn(xb)
    loss = loss_fn(pred, yb)
    optimizer.zero_grad() # torch uses grad from previous steps, this clear thos
    loss.backward()
    optimizer.step()

  if epoch % 100 == 0:
    print(f"Epoch {epoch} Loss {loss.item()}")
```

```
Epoch 0 Loss 0.7192122340202332
Epoch 100 Loss 0.694084107875824
Epoch 200 Loss 0.6825847625732422
Epoch 300 Loss 0.36202356219291687
Epoch 400 Loss 0.7643929719924927
Epoch 500 Loss 0.6928977966308594
Epoch 600 Loss 0.2199845314025879
Epoch 700 Loss 0.09722886979579926
Epoch 800 Loss 0.6950525045394897
Epoch 900 Loss 0.1731695979833603
Epoch 1000 Loss 0.04051987826824188
Epoch 1100 Loss 0.03799089789390564
Epoch 1200 Loss 0.0015338592929765582
Epoch 1300 Loss 0.0192149318754673
Epoch 1400 Loss 0.003236100310459733
```

In [35]:
```
Xte = torch.tensor([encode(t) for t,_ in test])
yte = torch.tensor([l for _,l in test])

rnn.eval()
with torch.no_grad():
  acc = rnn(Xte.to(device)).argmax(dim=1).eq(yte.to(device)).sum().item() / len(
print(acc)
```

```
0.6265
```